

UNIVERSIDAD PRIVADA DEL NORTE

Facultad de Ingeniería de sistemas Computacionales

T1 – Primera Evaluación: Programación Orientada a Objetos

Integrantes:

- Caceres Cotaquispe Keber Royer – N00224396
- Lima Jiménez Waldir - N00431592
- Reginaldo Paucar Ángel Eduardo - N00458156
- Meléndez Silva Jhojan Francis - N00311207
- Candro Juli Juan - N00486838

Docente: Torres Rodríguez Martin Eduardo



Lima, 2026

1. Introducción

La Programación Orientada a Objetos (POO) es uno de los paradigmas de programación más utilizados en el desarrollo de software moderno. Su principal finalidad es representar elementos del mundo real mediante objetos que poseen atributos y comportamientos específicos.

La POO facilita el diseño de aplicaciones escalables, mantenibles y reutilizables, permitiendo a los desarrolladores construir sistemas complejos mediante componentes independientes. Entre sus pilares fundamentales se encuentran el encapsulamiento, la herencia, el polimorfismo y la abstracción.

El presente informe analiza cuatro casos prácticos desarrollados en Java, los cuales permiten comprender y aplicar los principales conceptos de la Programación Orientada a Objetos dentro de un entorno académico y profesional.

2. Objetivos

1) Objetivo General

- Analizar y aplicar los principios fundamentales de la Programación Orientada a Objetos mediante la implementación de casos prácticos desarrollados en Java.

2) Objetivos Específicos

- Comprender el uso de clases y objetos.
- Aplicar el encapsulamiento mediante atributos privados.
- Implementar métodos estáticos y manejo de excepciones.
- Elaborar análisis de requerimientos utilizando UML.
- Utilizar herencia y polimorfismo para reutilizar código.
- Evaluar las ventajas de la Programación Orientada a Objetos en el desarrollo de software.

3. Marco Teórico

1. Programación Orientada a Objetos

La Programación Orientada a Objetos es un paradigma basado en la creación de objetos que interactúan entre sí para resolver problemas específicos.

2. Principales características

• Encapsulamiento

Permite ocultar los datos internos de una clase y controlar su acceso mediante métodos públicos.

• Herencia

Permite que una clase herede atributos y métodos de otra clase.

Polimorfismo

Permite que diferentes objetos respondan de manera distinta a una misma operación.

- **Abstracción**

Consiste en representar únicamente las características esenciales de un objeto.

4. **CASO 1: FUNDAMENTOS DE POO**

- **Descripción**

Se desarrolló una clase denominada Estudiante con atributos privados:

- Nombre
- Edad
- Carrera

Además, se implementaron:

- Constructor
- Métodos Getter
- Métodos Setter
- Método mostrarInformacion()

Posteriormente se crearon tres objetos para verificar el correcto funcionamiento de la clase.

- **Análisis**

Este caso demuestra la aplicación del encapsulamiento, uno de los principios fundamentales de la Programación Orientada a Objetos.

Al declarar los atributos como privados se evita la manipulación directa de la información, garantizando mayor seguridad y control sobre los datos almacenados.

Los métodos Getter y Setter permiten acceder y modificar los atributos de manera controlada, favoreciendo la integridad de la información.

- **Ventajas observadas**

- Protección de datos.
- Mayor mantenibilidad. Mejor organización del código.
- Reutilización de componentes.

5. **CASO 2: MÉTODOS STATIC Y EXCEPCIONES**

- **Descripción**

Se implementó una clase CuentaBancaria que contiene:

- Saldo privado.

- Método depositar().
- Método retirar().
- Contador static de cuentas creadas.
- Manejo de excepciones.
- Análisis

La palabra reservada static permite que una variable pertenezca a la clase y no a una instancia específica.

En este caso, el contador de cuentas puede ser compartido por todos los objetos creados.

Por otro lado, las excepciones permiten controlar errores durante la ejecución del programa.

Ejemplos de validación

- Evitar depósitos negativos.
- Evitar retiros superiores al saldo.
- Detectar entradas inválidas.
- Beneficios
- Mayor robustez.
- Prevención de errores.
- Mejor experiencia para el usuario.
- Incremento de la confiabilidad del sistema.

6. CASO 3: UML Y ANÁLISIS DE REQUERIMIENTOS

• Descripción

Se desarrolló un análisis previo del sistema mediante:

- Requerimientos funcionales.
- Requerimientos no funcionales.
- Historias de usuario.
- Diagrama de clases UML.

• Importancia del análisis

El análisis constituye una de las etapas más importantes del desarrollo de software porque permite comprender las necesidades del cliente antes de iniciar la programación.

• Requerimientos funcionales

Describen las acciones que debe realizar el sistema.

Ejemplos:

- Registrar estudiantes.

- Buscar información.
- Actualizar datos.
- **Requerimientos no funcionales**

Definen atributos de calidad.

Ejemplos:

- Seguridad.
- Rendimiento.
- Disponibilidad.
- Usabilidad.
- Historias de usuario

Las historias de usuario permiten identificar necesidades reales desde la perspectiva del usuario final.

Ejemplo:

- "Como administrador deseo registrar estudiantes para mantener actualizada la base de datos."

7. **CASO 4: HERENCIA Y POLIMORFISMO**

- **Descripción**

Se implementó una clase base denominada Persona y dos clases derivadas:

- Estudiante
- Docente

- **Herencia**

La herencia permite reutilizar atributos y métodos de una clase padre.

- **Beneficios**

- Reducción de código duplicado.
- Mayor organización.
- Facilidad de mantenimiento.

- **Polimorfismo**

El polimorfismo permite que distintos objetos respondan de manera diferente a un mismo método.

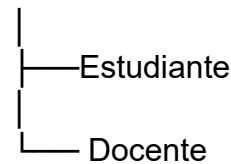
Por ejemplo:

`mostrarDatos()`

puede presentar información distinta dependiendo si el objeto corresponde a un estudiante o a un docente.

Aplicación práctica

Persona



Esta estructura facilita futuras ampliaciones del sistema sin modificar el código existente.

8. Resultados Obtenidos

Después de desarrollar los cuatro casos prácticos se obtuvieron los siguientes resultados:

- Comprensión del paradigma orientado a objetos.
- Aplicación efectiva del encapsulamiento.
- Implementación de validaciones mediante excepciones.
- Uso correcto de atributos y métodos estáticos.
- Modelado de sistemas mediante UML.
- Aplicación de herencia y polimorfismo.
- Desarrollo de soluciones más organizadas y reutilizables.

9. Conclusiones

- La Programación Orientada a Objetos permite desarrollar software más estructurado y mantenible.
- El encapsulamiento protege la información y mejora la seguridad de los datos.
- Los constructores facilitan la correcta inicialización de los objetos.
- Los métodos static permiten compartir información común entre todas las instancias de una clase.
- Las excepciones incrementan la robustez y confiabilidad de los programas.
- El análisis de requerimientos reduce errores durante el desarrollo.
- UML facilita la representación gráfica de sistemas orientados a objetos.
- La herencia favorece la reutilización de código.
- El polimorfismo incrementa la flexibilidad de las aplicaciones.
- Java proporciona herramientas adecuadas para implementar soluciones orientadas a objetos de manera eficiente.

10. Recomendaciones

- Aplicar siempre los principios de la Programación Orientada a Objetos.
- Utilizar UML antes de iniciar el desarrollo.
- Implementar validaciones mediante excepciones.

- Mantener una correcta documentación del código.
- Utilizar nombres descriptivos para clases y métodos.
- Seguir estándares de codificación para facilitar el mantenimiento.

11. Referencias Bibliográficas

Deitel, P., & Deitel, H. (2023). Java: How to Program. Pearson.

Sierra, K., & Bates, B. (2022). Head First Java. O'Reilly Media.

Oracle Corporation. Java Documentation.

<https://docs.oracle.com>

Pressman, R. (2020). Ingeniería del Software. McGraw-Hill.

Sommerville, I. (2021). Software Engineering. Pearson Education.